

Project Report
On
Hospital Patient Management System



UNIVERSITY INSTITUTE *of*
COMPUTING
Asia's Fastest Growing University

Program Name: BCA (Data Science)
Subject: Database Management System
(24CAT-203)

Submitted by : Harshita

UID: 24BCD10036

Section: 24BCD-1(A)

Submitted to: Mr. Suraj Parkash

Teacher's Signature:

Aim of the project :

The aim of the Hospital Patient Management System is to develop a well-structured database that efficiently manages hospital information such as patient details, doctor records, appointments, and medical reports. It focuses on ensuring accurate data storage, quick retrieval, and maintaining data integrity while reducing manual work and improving the overall efficiency of hospital operations.

Scope :

This project covers the management of patient and doctor information, scheduling of appointments, and maintaining medical reports. It is designed to be scalable for any healthcare setup, from small clinics to large hospitals, ensuring secure and consistent handling of healthcare data.

Objectives :

- To maintain patient details, doctor information, and appointment schedules.
- To store and retrieve medical records efficiently.
- To reduce paperwork and improve hospital workflow.
- To support quick data retrieval using SQL queries.
- To maintain data integrity using relational constraints and normalization.

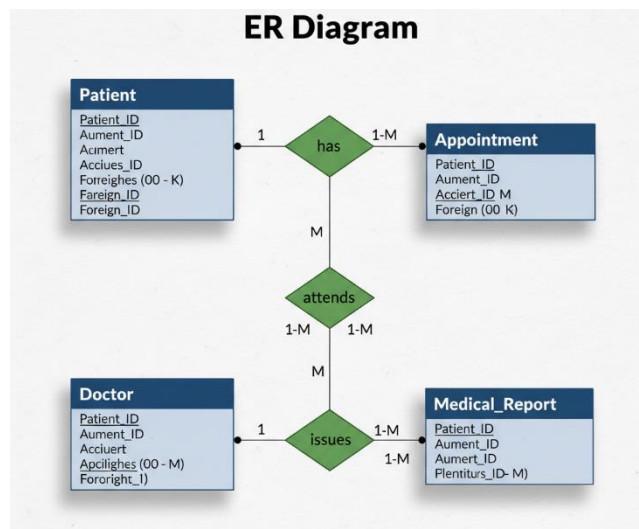
Phase 1 – Problem Identification

Entities and Relationships

- 1) **Patient** (Patient_ID, Name, Age, Gender, Contact, Address)
- 2) **Doctor** (Doctor_ID, Name, Specialization, Contact, Department)
- 3) **Appointment** (Appointment_ID, Patient_ID, Doctor_ID, Date, Time, Status)
- 4) **Medical_Report** (Report_ID, Patient_ID, Doctor_ID, Diagnosis, Prescription, Date)

Relationships:

- A **Patient** can have many **Appointments**.
- A **Doctor** can attend many **Appointments**.
- A **Patient** can have multiple **Medical Reports** issued by Doctors.



Phase 2 – Database Design

Relational Schema

After identifying the entities and relationships in the system, the next step is to convert the E–R diagram into a relational schema. The schema represents how the data will be stored in relational tables within the database.

- i. Patient (Patient_ID PK, Name, Age, Gender, Contact, Address)
- ii. Doctor (Doctor_ID PK, Name, Specialization, Contact, Department)
- iii. Appointment (Appointment_ID PK, Patient_ID FK, Doctor_ID FK, Date, Time, Status)
- iv. Medical_Report (Report_ID PK, Patient_ID FK, Doctor_ID FK, Diagnosis, Prescription, Date)

Each table contains attributes that describe real-world entities, and foreign keys establish relationships between them. For example, **Patient_ID** in the *Appointment* and *Medical_Report* tables connects patient information with their appointments and reports.

Normalization (Up to 3NF)

To ensure data consistency and eliminate redundancy, all tables are normalized up to the **Third Normal Form (3NF)**:

- **1NF:** Each table has atomic (indivisible) values, and every column holds only a single type of data.
- **2NF:** All non-key attributes are fully dependent on the primary key (no partial dependency).
- **3NF:** No transitive dependency exists — non-key attributes depend only on the primary key.

Thus, the database structure is efficient, consistent, and supports easy data maintenance.

Table Constraints

To maintain data integrity and enforce rules, several SQL constraints are applied during table creation:

- **PRIMARY KEY:** Ensures each record is uniquely identifiable.
 - **FOREIGN KEY:** Establishes relationships between tables (e.g., linking patients and doctors to appointments).
 - **NOT NULL:** Ensures mandatory fields like Name or Contact are not left empty.
 - **UNIQUE:** Prevents duplicate entries (e.g., Contact number).
-

Tools and Technologies Used

- **Database:** MySQL / Oracle
 - **Frontend (Optional):** HTML / PHP / Python (if interface added)
 - **Backend Language:** SQL
 - **Diagram Tool:** Draw.io / Lucidchart for ER Diagram
-

System Requirements

Hardware:

- Processor: Intel i3 or above
- RAM: 4GB minimum
- Storage: 500GB HDD

Software:

- MySQL / Oracle Database
- MySQL Workbench or SQL*Plus
- Windows / Linux OS

Phase 3 – Implementation & Queries

Once the database design was finalized, the next step was to implement the design using SQL commands. This phase includes creating database tables, inserting data, and writing various SQL queries for data retrieval and management.

1. DDL (Data Definition Language) Operations

These commands are used to define and modify the structure of database objects such as tables.

```
CREATE TABLE Patient (  
    Patient_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age > 0),  
    Gender CHAR(1),  
    Contact VARCHAR(15) UNIQUE,  
    Address VARCHAR(100)  
);
```

```
CREATE TABLE Doctor (  
    Doctor_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Specialization VARCHAR(50),  
    Contact VARCHAR(15),  
    Department VARCHAR(50)  
);
```

```
CREATE TABLE Appointment (  
    Appointment_ID INT PRIMARY KEY,  
    Patient_ID INT,  
    Doctor_ID INT,  
    Date DATE,  
    Time TIME,  
    Status VARCHAR(20),  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),  
    FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID)  
);
```

```
CREATE TABLE Medical_Report (  
    Report_ID INT PRIMARY KEY,  
    Patient_ID INT,  
    Doctor_ID INT,  
    Diagnosis VARCHAR(100),  
    Prescription VARCHAR(100),  
    Date DATE,  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),  
    FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID)  
);
```

Explanation:

- These statements create four tables — *Patient*, *Doctor*, *Appointment*, and *Medical_Report*.
- Each table includes **Primary Keys (PK)** for unique identification and **Foreign Keys (FK)** for maintaining relationships.
- Constraints such as **NOT NULL**, **CHECK**, and **UNIQUE** ensure data validity and integrity.

2. DML (Data Manipulation Language) Operations

These commands are used to insert, update, delete, and view data stored in the tables.

```
43 • INSERT INTO Patient
44   VALUES (1, 'Ravi Kumar', 32, 'M', '9876543210', 'Chandigarh'),
45          (2, 'Neha Sharma', 26, 'F', '9876501234', 'Mohali');
46 • SELECT * FROM Patient;
47
```

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:
Patient_ID	Name	Age	Gender	Contact	Address	
1	Ravi Kumar	32	M	9876543210	Chandigarh	
2	Neha Sharma	26	F	9876501234	Mohali	
*	NULL	NULL	NULL	NULL	NULL	

Patient Table Operations

- **Data Insertion:** Lines 55-57 insert two appointment records, linking Patient_ID 1 with Doctor_ID 101 and Patient_ID 2 with Doctor_ID 102. Line 58 then retrieves and displays all columns and rows that are currently stored in the **Appointment** table.
- **Output Validation:** The subsequent SELECT * FROM Patient query confirms that the two records were successfully inserted, displaying all attributes for both patients in the result grid.

- **INSERT INTO** Doctor
- **VALUES** (101, 'Dr. Mehta', 'Cardiology', '9998877665', 'Cardio Dept'),
- (102, 'Dr. Kapoor', 'Dermatology', '9988776655', 'Skin Dept');
- **SELECT * FROM** Doctor;

Doctor_ID	Name	Specialization	Contact	Department
101	Dr. Mehta	Cardiology	9998877665	Cardio Dept
102	Dr. Kapoor	Dermatology	9988776655	Skin Dept
NULL	NULL	NULL	NULL	NULL

Doctor Table Operations

- **Data Insertion:** Lines 49-51 insert two rows into the **Doctor** table, adding 'Dr. Mehta' (ID 101) and 'Dr. Kapoor' (ID 102) with their specializations. Line 52 then retrieves and displays all columns and rows that are currently stored in the **Doctor** table.
- **Output Validation:** The subsequent **SELECT * FROM Doctor** query verifies the successful insertion, listing both doctors along with their specializations, contacts, and departments in the result grid.

- 55 • **INSERT INTO** Appointment
- 56 **VALUES** (5001, 1, 101, '2025-03-05', '10:30:00', 'Confirmed'),
- 57 (5002, 2, 102, '2025-03-06', '11:00:00', 'Pending');
- 58 • **SELECT * FROM** Appointment;
- 59

Appointment_ID	Patient_ID	Doctor_ID	Date	Time	Status
5001	1	101	2025-03-05	10:30:00	Confirmed
5002	2	102	2025-03-06	11:00:00	Pending
NULL	NULL	NULL	NULL	NULL	NULL

Appointment Table Operations

- **Data Insertion:** Lines 55-57 insert two appointment records, linking Patient_ID 1 with Doctor_ID 101 and Patient_ID 2 with Doctor_ID 102. Line 58 then retrieves and displays all columns and rows that are currently stored in the **Appointment** table.
- **Output Validation:** The subsequent SELECT * FROM Appointment query shows that the two appointments were recorded correctly, displaying the Appointment_ID, linked Patient_ID and Doctor_ID, and the correct **Status** for each record.

```

61 • INSERT INTO Medical_Report
62     VALUES (9001, 1, 101, 'High BP', 'Take medicine daily', '2025-03-05'),
63     (9002, 2, 102, 'Skin Allergy', 'Use prescribed cream', '2025-03-06');
64 • SELECT * FROM Medical_Report;
65

```

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content: 
Report_ID	Patient_ID	Doctor_ID	Diagnosis	Prescription	Date	
9001	1	101	High BP	Take medicine daily	2025-03-05	
9002	2	102	Skin Allergy	Use prescribed cream	2025-03-06	
NULL	NULL	NULL	NULL	NULL	NULL	

Medical_Report Table Operations

- **Data Insertion:** Lines 61-63 insert two medical reports, linking Report_ID 9001 to Patient 1 and Doctor 101, and Report_ID 9002 to Patient 2 and Doctor 102. Line 64 then retrieves and displays all columns and rows that are currently stored in the **Medical_Report** table.
- **Output Validation:** The subsequent SELECT * FROM Medical_Report query confirms the successful creation of both reports, correctly linking them to the Patient_ID and Doctor_ID and displaying the Diagnosis and Prescription details.

3. DQL (Data Query Language) – Data Retrieval Queries

Below are some example queries used for retrieving data and generating useful reports.

```
68 -- 1. Display all patients
```

```
69 • SELECT * FROM Patient;
```

```
70
```

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
	Patient_ID	Name	Age	Gender	Contact	Address
▶	1	Ravi Kumar	32	M	9876543210	Chandigarh
	2	Neha Sharma	26	F	9876501234	Mohali
*	NULL	NULL	NULL	NULL	NULL	NULL

```
73 -- 2. Show all appointments for a specific doctor
```

```
74 • SELECT P.Name AS Patient_Name, A.Date, A.Time, A.Status
```

```
75 FROM Appointment A
```

```
76 JOIN Patient P ON A.Patient_ID = P.Patient_ID
```

```
77 WHERE A.Doctor_ID = 101;
```

```
78
```

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
	Patient_Name	Date	Time	Status
▶	Ravi Kumar	2025-03-05	10:30:00	Confirmed

```
79 -- 3. Count total appointments for each doctor
```

```
80 • SELECT Doctor_ID, COUNT(*) AS Total_Appointments
```

```
81 FROM Appointment
```

```
82 GROUP BY Doctor_ID
```

```
83 HAVING COUNT(*) > 0;
```

```
84
```

Result Grid		
Filter Rows:		
Export:		
Wrap Cell Content:		
	Doctor_ID	Total_Appointments
▶	101	1
	102	1

```

85  -- 4. Display patients with their reports and doctor names
86 • SELECT P.Name AS Patient, D.Name AS Doctor, R.Diagnosis, R.Prescription
87 FROM Medical_Report R
88 JOIN Patient P ON R.Patient_ID = P.Patient_ID
89 JOIN Doctor D ON R.Doctor_ID = D.Doctor_ID;
90

```

Patient	Doctor	Diagnosis	Prescription
Ravi Kumar	Dr. Mehta	High BP	Take medicine daily
Neha Sharma	Dr. Kapoor	Skin Allergy	Use prescribed cream

```

91  -- 5. Create a view for patient details with medical reports
92 • CREATE VIEW Patient_Reports AS
93 SELECT P.Name, D.Name AS Doctor, R.Diagnosis, R.Prescription
94 FROM Patient P
95 JOIN Medical_Report R ON P.Patient_ID = R.Patient_ID
96 JOIN Doctor D ON D.Doctor_ID = R.Doctor_ID;
97
98 • SELECT * FROM Patient_Reports;
99

```

Name	Doctor	Diagnosis	Prescription
Ravi Kumar	Dr. Mehta	High BP	Take medicine daily
Neha Sharma	Dr. Kapoor	Skin Allergy	Use prescribed cream

Explanation:

- The **SELECT** statements retrieve and display data from the database.
- **JOIN** operations combine data from multiple tables to show relationships between patients, doctors, and reports.
- **GROUP BY** and **HAVING** are used for data summarization, such as counting appointments per doctor.
- The **VIEW** is created to simplify frequent queries and provide a combined display of key information.

Phase 4 – Presentation

This phase involves presenting the final database design, SQL queries, and output results to demonstrate the functionality of the **Hospital Patient Management System**. It also includes explaining the key design decisions, implementation steps, and the learning outcomes achieved throughout the project.

1. Presentation Overview

- The project was presented in four stages — Problem Identification, Database Design, Implementation, and Output Presentation.
- The **E–R Diagram** and **Relational Schema** were used to explain the structure and relationships among entities.
- **SQL queries** were demonstrated live to show how data can be inserted, retrieved, and managed efficiently.
- Screenshots of executed queries and their results were shown as proof of working implementation.

2. Design Decisions

- Entities were chosen based on real-world hospital operations (Patients, Doctors, Appointments, and Reports).
- Foreign Key relationships were established to maintain data consistency and prevent orphan records.
- Normalization up to **3NF** ensured reduced redundancy and improved data organization.
- Constraints such as **PRIMARY KEY**, **NOT NULL**, **CHECK**, and **UNIQUE** were applied to maintain data validity.
- A **View** was created to simplify access to frequently used combined data, improving query efficiency.

3. Learning Outcomes

- Gained a practical understanding of how to convert real-life requirements into a database design.
- Learned how to create and normalize tables to improve efficiency and avoid redundancy.
- Acquired experience with SQL commands (DDL, DML, DQL) and constraints.
- Understood how to apply **JOIN**, **GROUP BY**, **HAVING**, and **VIEW** for meaningful data retrieval.
- Learned to validate relationships through E–R Diagrams and Relational Schemas.
- Developed confidence in designing, implementing, and presenting a real-world DBMS project.

4. Skills Acquired

*Every line of code taught precision. Every query taught logic.
And every challenge taught patience and problem-solving.*

- Database design and ER diagram creation.
- Conversion of E–R model to relational schema.
- Implementation of constraints (PK, FK, UNIQUE, CHECK, NOT NULL).
- Writing and optimizing SQL queries (DDL, DML, DQL).
- Use of normalization techniques up to 3NF.
- Testing and validating database functionality.

✦✦ Beyond the Data — A Journey Through the Hospital Patient Management System

From scattered records to structured care, this project brings order to information and meaning to management!

Through this project, the boundaries between *technology* and *healthcare* came together — showing how a few well-designed tables, relationships, and queries can make a huge difference in managing information that ultimately touches lives.

It reflects the importance of accuracy, organization, and empathy — because behind every data entry lies a person, a story, and a step toward better care.

✿ ***“In the world of databases, every field has meaning, every relation has purpose, and every well-designed system becomes a quiet form of service.”**

Working on this project was more than just learning SQL commands or E–R diagrams — it was about understanding how digital systems can bring real change. From designing the schema to writing the final query, every phase strengthened the bridge between logic and creativity, precision and compassion.

*As the last line of code executes and the final report closes,
this project remains a reminder that learning is not about completion — it’s about creation. ✨*

🌹 Thank You for Reading

With this, the journey of building the Hospital Patient Management System comes to an end — but the learning it brought will continue to inspire future creations.